applied sciences

IMPACT FACTOR 2.5

CITESCORE 5.5

Article

Efficient Task Allocation in Multi-Agent Systems Using Reinforcement Learning and Genetic Algorithm

Zheng Fang, Tao Ma, Jun Huang, Zhao Niu and Fang Yang

MDPI

https://doi.org/10.3390/app15041905

applied sciences

MDPI

Article

Efficient Task Allocation in Multi-Agent Systems Using Reinforcement Learning and Genetic Algorithm

Zheng Fang , Tao Ma *, Jun Huang, Zhao Niu and Fang Yang

College of Electronic Engineering, National University of Defense Technology, Hefei 230037, China; fangzhen@nudt.edu.cn (Z.F.); huangjun0111@nudt.edu.cn (J.H.); niuzhao@nudt.edu.cn (Z.N.); yangfang@nudt.edu.cn (F.Y.)

* Correspondence: matao@nudt.edu.cn

Abstract: The multi-agent task allocation problem has attracted substantial research interest due to the increasing demand for effective solutions in large-scale and dynamic environments. Despite advancements, traditional algorithms often fall short in optimizing efficiency and adaptability within complex scenarios. To address this shortcoming, we propose a genetic algorithm-enhanced PPO (GAPPO) algorithm, specifically developed to enhance decision-making in challenging task allocation contexts. GAPPO employs a deep reinforcement learning framework, enabling each agent to independently evaluate its surroundings, manage energy resources, and adaptively adjust task allocations in response to evolving conditions. Through iterative refinement, GAPPO achieves balanced task distribution and minimizes energy consumption across diverse configurations. Comprehensive simulations demonstrate that GAPPO consistently outperforms traditional algorithms, resulting in reduced task completion time and heightened energy efficiency. Our findings underscore GAPPO's potential as a robust solution for real-time multi-agent task allocation.

Keywords: multi-agent systems; task allocation; reinforcement learning; genetic algorithm

check for updates

Academic Editor: Christos Bouras

Received: 19 January 2025
Revised: 9 February 2025
Accepted: 11 February 2025
Published: 12 February 2025

Citation: Fang, Z.; Ma, T.; Huang, J.; Niu, Z.; Yang, F. Efficient Task Allocation in Multi-Agent Systems Using Reinforcement Learning and Genetic Algorithm. *Appl. Sci.* **2025**, *15*, 1905. https://doi.org/10.3390/app15041905

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The multi-agent task allocation problem involves distributing tasks among multiple agents to optimize resource utilization and minimize operational costs [1,2]. Effective coordination among agents is essential to enhance performance across a range of applications [3,4]. Effective task allocation is crucial in these contexts, as it directly impacts the system's overall performance, efficiency, and success [5]. The complexity of this problem lies in balancing task requirements, agent capabilities, and environmental constraints, each of which can vary significantly [6]. Developing a robust allocation scheme is crucial to avoid resource wastage and to ensure tasks are assigned to the most suitable agents. As the scale of agents and tasks increases, computational and logistical demands rise, emphasizing the need for adaptive and efficient solutions [7]. In recent years, multi-agent systems have shown significant potential in applications requiring rapid deployment and dynamic task allocation, such as agriculture [8,9], intelligent transportation [10,11], and disaster response—including rescue [12], search [13], tracking operations [14,15], etc. [16].

Existing algorithms for multi-agent task allocation primarily fall into centralized and distributed frameworks. Centralized algorithms leverage global information for optimization but struggle with scalability and adaptability in dynamic environments. In contrast, distributed algorithms enhance flexibility by allowing agents to make decisions based on local observations, though they often result in suboptimal allocations due to limited coordination. Reinforcement learning (RL) has gained attention as an alternative [17,18],

enabling agents to learn allocation strategies through interaction with the environment. However, conventional RL algorithms often suffer from slow convergence and instability, particularly in large-scale and dynamic settings [19,20].

The multi-agent task allocation problem considered in this work is computationally challenging due to its combinatorial nature. The problem requires determining an optimal task allocation policy while considering constraints such as agent capabilities, dynamic task demands, and communication limitations. Similar combinatorial optimization problems, such as the multi-agent traveling salesman problem (mTSP) and job-shop scheduling, have been shown to be NP-hard [21]. Given the exponential growth of the solution space as the number of agents and tasks increases, solving this problem optimally in real time becomes computationally infeasible with traditional optimization techniques.

To overcome those limitations, we propose a novel algorithm that integrates genetic algorithms (GAs) with proximal policy optimization (PPO) to enhance task allocation efficiency in multi-agent systems. The GA facilitates exploration through selection, crossover, and mutation, improving policy diversity and search effectiveness. Meanwhile, PPO optimizes decision-making by leveraging real-time feedback, enabling agents to dynamically adjust allocations. This hybrid algorithm enhances adaptability and learning efficiency in dynamic task allocation scenarios. By combining the GA's exploratory strengths with PPO's optimization capabilities, our algorithm addresses the challenges posed by traditional RL algorithms and provides a more scalable solution to the task allocation problem.

The main contributions of this paper are as follows.

1. We model the multi-agent task allocation problem as a Markov game, where each agent functions as an independent agent, interacting continuously with its environment to iteratively improve its task allocation. Under this framework, each agent determines its actions based on local observations, facilitating coordinated and efficient task distribution across the swarm.
2. We propose a GA-PPO reinforcement learning algorithm, which combines genetic algorithms with proximal policy optimization, incorporating an attention mechanism and adaptive learning rate. This algorithm empowers each agent to autonomously adapt to changing task demands, improving learning efficiency and facilitating effective coordination with multi-agent environments. By leveraging these features, the system optimizes resource utilization while maintaining robust performance in dynamic scenarios.
3. Numerical experiments demonstrate that our proposed algorithm performs well in terms of convergence speed and scalability compared with the existing representative algorithms.

This paper is organized as follows. Section 2 reviews related work. Section 3 describes the problem formulation. Section 4 presents the GAPPO algorithm and provides its detailed process. Section 5 discusses the performance of the algorithm in different environments. Section 6 presents the summary and future directions. The main notations of this paper are listed in Table 1.

Table 1. Notation used in this paper.

Notation	Definitions
$n(t)$	the number of agents at time t
m	the number of tasks
N	the set of $n(t)$ agents
S	the set of m tasks
N_j	the number of agents participating in task j
$\{S\}_i$	the task that agent i chooses

Table 1. *Cont.*

Notation	Definitions
$\{I\}_j$	the set of agents or the group for task j
ω_i	the work ability of agent i
h_j	the workload of task j
T_j	the completion time of task j
T	the max completion time of all tasks
A_i	the strategy set of agent i
A	the strategy space
τ	the discount factor reflecting the rate of task reward change
Rad_i	the communication radius of agent i j
R_j	the reward of task j
E_i	the battery level of the agent i
v_i	the speed of the agent i
θ	the elevation angle of the agent

2. Related Work

2.1. Centralized Optimization Algorithms

Many traditional algorithms rely on centralized optimization algorithms, leveraging global information to optimize task allocation. These algorithms often employ mixed-integer programming [22] or centralized RL models [23], achieving high-quality solutions under well-structured environments.

For instance, Shabanighazikelayeh et al. [24] proposed a centralized algorithm for UAV placement under high-altitude constraints, ensuring efficient coverage and communication. Similarly, Consul et al. [25] applied a hybrid federated RL framework in a UAV-assisted mobile edge computing (MEC) network, improving computation offloading and resource allocation. These algorithms demonstrate that centralized optimization achieves high efficiency in controlled settings with predictable task demands.

However, centralized algorithms face scalability and adaptability challenges. Wang et al. [26] highlighted that centralized algorithms optimize path planning effectively but struggle with rapid environmental changes, necessitating frequent re-optimization. Al-Hussaini et al. [27] proposed an automated task reallocation system for multi-robot missions, yet the algorithm remains constrained by centralized control bottlenecks. Furthermore, centralized algorithms introduce single points of failure, increasing system vulnerability in large-scale and dynamic environments [28].

Comparison with GA-PPO: Unlike centralized algorithms that rely on a single controller, GA-PPO employs a decentralized framework, where genetic algorithms enhance exploration, and PPO dynamically refines policies. Compared to Shabanighazikelayeh et al.'s [24] UAV optimization algorithm, GA-PPO does not require predefined constraints, allowing greater adaptability to changing conditions. Furthermore, while Consul et al.'s [25] federated RL algorithm improves computational efficiency, it still relies on a central coordinator. GA-PPO eliminates this dependency, making it more robust to network failures and large-scale applications.

2.2. Distributed Optimization Algorithms

Distributed optimization algorithms decentralize decision-making, allowing agents to allocate tasks based on local information with limited communication. This structure enhances scalability, adaptability, and fault tolerance by enabling agents to act independently, without the need for global communication [29]. Ren et al. [30] demonstrated the effectiveness of distributed optimization in large-scale wireless sensor networks, where agents can allocate resources based on local observations, thus improving system efficiency.

Luo et al. [31] applied similar principles to multi-robot systems, optimizing task allocation to increase operational efficiency while maintaining system scalability.

Despite these advantages, distributed optimization faces challenges in terms of coordination and convergence. Zhang et al. [32] highlighted inefficiencies arising from incomplete information, where agents' local knowledge may lead to suboptimal allocations. Moreover, the absence of global coordination can cause slower convergence, as agents may not be able to adjust their strategies based on the broader system dynamics [33]. These limitations can result in inefficiencies, particularly in dynamic environments where global coordination is beneficial for maintaining optimal system performance.

Comparison with GA-PPO: GA-PPO overcomes many of these challenges through the integration of genetic algorithms for broad exploration and proximal policy optimization for continuous policy refinement. This combination allows GA-PPO to adapt dynamically to changing environments, ensuring both efficient exploration and targeted optimization over time. Unlike Ren et al.'s [30] distributed algorithm, which relies heavily on local communication between agents, GA-PPO does not require fixed communication patterns, providing more flexibility and robustness to network failures. Furthermore, while Luo et al.'s [31] work in multi-robot systems benefits from localized decision-making, GA-PPO's hybrid framework allows it to perform effectively even in scenarios with high levels of environmental uncertainty and variable task demands, making it more adaptable and scalable in comparison.

2.3. Reinforcement Learning-Based Algorithms

Reinforcement learning (RL) has emerged as a powerful algorithm for multi-agent task allocation, allowing agents to adapt and optimize task distribution strategies through real-time interactions with dynamic environments. Its ability to manage complex, evolving scenarios and enhance decision-making processes makes it especially suitable for multi-agent systems [34,35].

For instance, Wu et al. proposed a multi-agent deep reinforcement learning algorithm for formation control, which integrates an attention mechanism and adaptive accuracy to enhance agent coordination and precision [5]. Similarly, Ning et al. developed a joint optimization algorithm for data acquisition and trajectory planning in UAV-assisted IoT networks. Their algorithm maximizes energy efficiency while adhering to mobility, safety, and task constraints, framing the problem as a constrained Markov decision process, and using multi-agent deep RL to optimize agent movement policies [36]. These algorithms demonstrate the potential of RL in optimizing agent behaviors for specific applications with controlled constraints.

Xu et al. introduced a federated deep RL algorithm for agent deployment and resource allocation in 6G networks, enabling agents to make real-time decisions based on local observations while pursuing a global optimal solution [37]. Their algorithm significantly improves network throughput and convergence, as demonstrated in simulations. Moreover, Dai et al. developed a multi-agent collaborative RL algorithm for agent resource allocation in wireless networks, focusing on optimizing interference management and network capacity. This framework integrates federated learning to facilitate data sharing and cooperation among agents, outperforming traditional RL algorithms in simulation [23].

Comparison with GA-PPO: Unlike the aforementioned RL-based algorithms that rely on centralized or federated frameworks, GA-PPO employs a decentralized algorithm, offering improved scalability and robustness. While Wu et al.'s [5] algorithm enhances coordination through an attention mechanism, GA-PPO's use of genetic algorithms for exploration and proximal policy optimization (PPO) for dynamic policy refinement provides greater flexibility and adaptability. This allows GA-PPO to adjust in real time without

the need for centralized control. In comparison to Ning et al.'s [36] constrained Markov decision process, which is limited to specific task constraints, GA-PPO does not require predefined constraints, enabling it to operate across a wider range of environments. Furthermore, while Xu et al.'s [37] federated RL algorithm relies on local coordination between agents, GA-PPO eliminates the need for central coordination, making it more robust to network failures. This decentralized nature makes GA-PPO particularly well suited for large-scale applications, where adaptability and resilience are critical.

3. Problem Formulation

In this paper, we address the multi-agent task allocation problem, where agents are assigned to distinct tasks and operate either independently or cooperatively to enhance task efficiency and coverage across the environment.

More specifically, agents have the following two tasks.

1. Primary (Extrinsic) Task: High-quality execution of multiple operational tasks by agents, such as conducting surveillance on designated targets or performing search and rescue missions in critical scenarios. Those tasks necessitate that agents autonomously assess their environments and adapt to dynamic conditions, thereby ensuring effective and efficient performance.
2. Secondary (Intrinsic) Task: Evaluation of task allocation decisions. This task focuses on evaluating the effectiveness of agents' decisions in optimizing resource utilization and enhancing operational efficiency, thereby improving the overall performance of the multi-agent system.

To achieve these objectives, which include the high-quality execution of operational tasks and the evaluation of task allocation strategies, each agent undertakes the following steps: state estimation, task allocation (for scenarios involving multiple tasks), and strategy, as illustrated in Figure 1 and elaborated upon in the subsequent sections.

1. State Estimation: Each agent's state includes its position, an occupancy map, and target identifiers. At time t, the state S_{ti} reflects its configuration and location, updated in real time. The transition function $f(S_{ti}, a_i)$ models state evolution. The Actor–Critic network processes these data to optimize policies, enabling efficient navigation, obstacle avoidance, and target acquisition in dynamic environments.
2. Task Allocation: A dynamic algorithm optimizes task distribution, preventing clustering. Agents evaluate tasks based on proximity, urgency, and workload, considering completion time and expected benefits. They share status updates for real-time reallocation, ensuring balanced task coverage and efficient resource use.
3. Strategy: Agents optimize detection and mapping based on state estimates. Initially, they update policies independently via rewards. In collaborative settings, they share policy gradients to refine strategies. GAPPO, with attention mechanisms, enables autonomous and cooperative decision-making for better coordination.

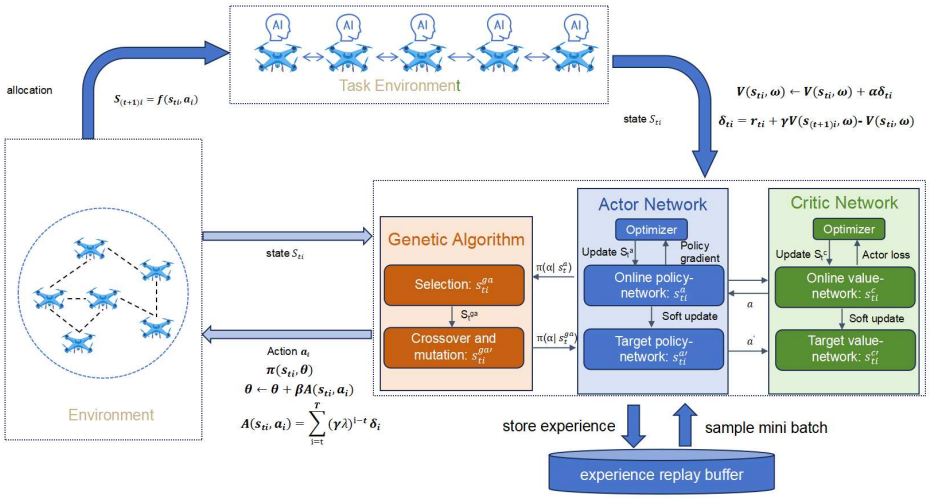


Figure 1. Adaptive task-oriented GA-PPO for agent swarms algorithm.

3.1. System Model

Suppose there exist a set of agents formed by $N = \{1, 2, \ldots, n\}$ and a set of tasks formed by $S = \{1, 2, \ldots, m\}$. An agent cannot participate in multiple tasks simultaneously. The task allocation problem in this multi-agent system can be formulated by defining the strategy space for each agent. Let $\{S\}_i$ represent the task selected by agent $i \in N$, and let $\{I\}_j$ represent the set of agents allocated to task $j \in S$. Each task j is positioned at a fixed location $pos_j = (x_j, y_j, z_j)$, defined by its three-dimensional coordinates. The position of agent i, pos_i, coincides with the position of the task it selects. The Euclidean distance between agent i and task j is expressed as

$$d_{ij} = \|\text{pos}_i - \text{pos}_j\|. \tag{1}$$

Here, d_{ij} also represents a constraint on the communication range of the agents. Specifically, an agent can only access tasks or interact with other agents that are within its communication range, as determined by d_{ij}.

The workload of task j, represented as $h_j \geq 0$, represents the total effort required to complete the task through the combined efforts of the agents. The work ability of agent i is represented by ω_i, indicating the amount of work that agent i can execute per unit time. For task j, the work ability of the group allocated to the task is defined as the sum of the work capacities of all agents in that group $\sum_{i \in I_j} \omega_i$. To complete task j, the total work ability of the allocated group must satisfy

$$\sum\nolimits_{i \in \{I\}_j} \omega_i - h_j \geq 0. \tag{2}$$

For each task $j \in S$, it is crucial to allocate an appropriate number of agents to satisfy (2) and thereby minimize resource wastage due to inefficient allocations. Thus, the reward for task j is expressed as

$$R_j = -\tau^{\sum_{i \in \{I\}_j} \omega_i - h_j}, \tag{3}$$

where discount factor $\tau \in (0, 1)$ represents the rate of change in reward based on the work ability of the group $\{I\}_j$. It also indicates the timeliness of the task. When τ approaches one, the reward continues to increase even if (2) is satisfied, encouraging more agents to participate and thereby speeding up task completion. Conversely, when τ approaches 0, the reward R_j becomes insensitive to the (2), meaning no additional agents would gain rewards from the task once the condition is satisfied.

The objective of the task allocation problem is to identify an optimal allocation that minimizes the total completion time $T = \max\{T_1, T_2, \ldots, T_m\}$, where the completion time for task j is expressed as

$$T_j = \frac{h_j}{\sum_{i \in \{I\}_j} \omega_i}. \tag{4}$$

Therefore, the problem can be formulated as follows:

$$\begin{aligned} &\min_{\{e_{ij}\}} \{T_j\}, \\ &\text{s.t.} \quad \sum_{j=1}^{m} e_{ij} \leq 1, \quad \forall i \in N, \\ &e_{ij} \in \{0,1\}, \quad \forall i \in N, \forall j \in S, \end{aligned} \tag{5}$$

where e_{ij} is a binary decision variable, if $e_{ij} = 1$ indicates that agent i is allocated to task j; otherwise, agent i is not allocated to task j.

3.2. State Action Model

In our agent swarm framework, each agent's transition at time t is represented by the tuple (s_t, a_t, r_t, s_{t+1}), where the following hold:

- s_t denotes the agent's current state, which is a composite representation consisting of the following:
  - The agent's location $\text{pos}_i = (x_i, y_i, z_i)$, providing the agent's spatial position in the environment. This information is essential for determining the agent's proximity to tasks and other agents, which influences both task allocation and collision avoidance.
  - Task-related information, including task priorities, workload status, and the agent's assigned task. This information allows the agent to evaluate its current workload and adjust its decisions based on the urgency and importance of tasks within its scope.
  - The occupancy map, encoding the spatial distribution of tasks and agents within the agent's communication or sensing range. This map helps the agent understand its environment by providing real-time updates on the locations of other agents and tasks, enabling more informed decisions on task allocation and collaboration with other agents.
- a_t is the action selected by the agent in state s_t. The action is defined as the selection of a task $j \in S$, where S represents the set of available tasks. Formally,

$$a_t = \arg\max_{j \in \mathcal{C}(i)} \pi(s_t, j), \tag{6}$$

where $\pi(s_t, j)$ is the policy function that outputs the probability of selecting task j given the current state s_t, and $\mathcal{C}(i)$ is the set of tasks within the communication or sensing range of agent i. If no tasks are feasible, the agent may choose an idle action. The policy function $\pi(s_t, j)$ is learned using the Actor–Critic architecture, where the Actor is responsible for selecting actions based on the current state, while the Critic evaluates the actions taken by estimating the value function. Specifically, the Actor network takes the current state s_t as input and outputs a probability distribution over possible actions (tasks). The policy function $\pi(s_t, j)$ is expressed as

$$\pi(s_t, j) = P(a_t = j \mid s_t), \tag{7}$$

where a_t is the action selected by the agent at time step t, and j is a task from the set of available tasks $\mathcal{C}(i)$. The Critic, on the other hand, estimates the expected cumulative reward for the agent starting from state s_t. This is achieved through the value function $V(s_t)$, which represents the expected long-term reward given the current state. It is formally expressed as

$$V(s_t) = \mathbb{E}\left[\sum_{k=t}^{T} \gamma^{k-t} r_k \mid s_t\right], \tag{8}$$

where r_k is the reward received at time step k, and γ is the discount factor that determines the importance of future rewards.

At each time step, the agent selects an action a_t based on the policy $\pi(s_t, j)$ by maximizing the probability of selecting a task from the set $\mathcal{C}(i)$ based on the current state s_t. The interaction between the Actor and Critic helps in continuously optimizing the policy, with the Critic providing feedback on the expected rewards, guiding the Actor to improve its decision-making over time.

If no feasible tasks are within the agent's sensing or communication range, the agent may choose an idle action, which corresponds to not selecting any task. This idle action is implicitly represented within the action space, where one of the actions is designated as "idle".

- r_t represents the reward obtained after executing action a_t. This reward reflects the agent's contribution to task completion, considering factors such as task importance, completion effectiveness, and collaborative efficiency with other agents.
- s_{t+1} is the subsequent state resulting from the agent's interaction with the environment after performing a_t.

This process encapsulates the agents' decision-making as they interact with the environment, where each agent optimizes its actions to maximize cumulative rewards using a combination of Actor–Critic and genetic algorithms.

In the proximal policy optimization (PPO) framework, the advantage function $A(s_t, a_t)$ evaluates the relative benefit of selecting action a_t in state s_t. PPO employs Generalized Advantage Estimation (GAE) to compute the advantage. The advantage function is given by

$$A(s_t, a_t) = \hat{A}_t = \sum_{i=t}^{T} (\gamma\lambda)^{i-t} \delta_i, \tag{9}$$

where γ is the discount factor, which controls the importance of future rewards relative to immediate rewards. It is a value between 0 and 1. λ is the smoothing parameter, which helps balance the bias-variance trade-off in estimating the advantage. It is also between 0 and 1. δ_i is the temporal difference error (TD-error), which is expressed as

$$\delta_i = r_i + \gamma V(s_{i+1}; \omega) - V(s_i; \omega), \tag{10}$$

where γ is the discount factor. The value function $V(s_i; \omega)$ is updated iteratively as

$$V(s_i; \omega) \leftarrow V(s_i; \omega) + \alpha \delta_i, \tag{11}$$

where α is the learning rate. This update process ensures that each agent effectively optimizes its cumulative rewards, leading to more precise task execution and resource allocation. The initial value of $V(s_i; \omega)$ is typically set to zero.

Moreover, the GA enhances policy optimization across the agent swarm by selecting and evolving the most effective action strategies. Integrating the GA with the Actor–Critic framework inherent in the PPO algorithm achieves a refined balance between exploration and exploitation. Consequently, this framework significantly improves adaptability in dynamic task environments. The synergy between the GA and the Actor–Critic mechanism

of PPO not only fosters effective strategy development but also contributes to the overall robustness and efficiency of the agent swarm in complex operational scenarios.

4. A Learning Algorithm

There exist various optimization algorithms for addressing multi-agent task allocation problems, including the leader–follower algorithm [38], the virtual structure algorithm [39], and behavior-based control algorithm [40]. In addition, reinforcement learning has demonstrated effectiveness in this area. To address the challenges associated with dynamic task allocation among agents, this paper presents a novel algorithm, the genetic algorithm-enhanced proximal policy optimization (GA-PPO) algorithm, the aim of which is to enhance decision-making and coordination among agents in dynamic environments.

4.1. Genetic Algorithm-Enhanced Proximal Policy Optimization

This algorithm leverages the efficiency of PPO for learning optimal policies while utilizing the evolutionary principles of the GA to select the best-performing agents, thereby enhancing decision-making in dynamic environments.

The algorithm begins by initializing a population P of Actor–Critic networks, with each individual representing a distinct policy tailored for agent operations. Several key hyperparameters guide the learning process, including the learning rate lr, discount factor γ, clip ratio ϵ, and KL divergence target $\mathrm{KL}_{\mathrm{target}}$. Those values are selected based on theoretical foundations and established practices in reinforcement learning and evolutionary algorithms, ensuring a stable and efficient learning process. The specific values for these hyperparameters are chosen to align with common standards in the literature, which have been shown to perform well in similar multi-agent systems tasks.

A replay buffer with a capacity of C = 10,000 is created to store observed state–action–reward sequences. This buffer facilitates experience sampling during training. Additionally, the genetic algorithm component employs a set of parameters, including mutation rates and fitness weights (α, β, and γ), which govern the selection process and population evolution.

The learning rate lr determines the step size during policy updates, with a typical value in PPO implementations chosen to ensure stable convergence. The discount factor γ reflects the importance of future rewards, and it is set to a commonly used value for long-term decision-making tasks. The clip ratio ϵ is used to prevent excessive updates during the training process, ensuring stability while maintaining exploration. The KL divergence target $\mathrm{KL}_{\mathrm{target}}$ helps control the divergence between old and new policies, promoting gradual updates to the policy.

The genetic algorithm parameters, such as mutation rates and fitness weights, are designed to balance reward maximization, exploration, and resource efficiency. Those parameters are selected based on general practices in evolutionary algorithms, adapted for the task at hand. The fitness score of each individual is computed based on cumulative reward, policy entropy, and computational resource cost, with appropriate weights (α, β, and γ) to guide the optimization process.

The fitness score of each individual is computed based on cumulative reward, policy entropy, and computational resource cost, with appropriate weights (α, β, and γ) to guide the optimization process.

The selection of hyperparameters follows established guidelines in reinforcement learning and evolutionary algorithms. However, the exact values used may not be optimal across all scenarios. Specifically, the performance of the algorithm could benefit from fine-tuning the parameters to better adapt to varying environments and tasks. While the current work does not include a comprehensive optimization of these hyperparameters,

future research could involve systematically varying them to assess their influence on the performance of GAPPO.

During each iteration, the algorithm collects trajectories for each individual P_i with the population. Each agent observes the current state s_t and selects an action a_t based on its policy P_i. The selected action is executed, resulting in a reward r_t and a subsequent state s_{t+1}. The transition (s_t, a_t, r_t, s_{t+1}) is appended to the trajectory T, and this process continues until a terminal state is reached. The accumulated trajectories are stored in the replay buffer, allowing the PPO component to sample batches for optimization.

Once the replay buffer contains a sufficient number of samples, the strategy update phase commences. A batch of transitions is sampled, and for each transition, the algorithm computes the returns R_t using the discount factor γ. The advantages $\hat{A}_t$ are calculated utilizing generalized advantage estimation (GAE). The action probabilities $\pi(a_t|s_t)$ and state values $V(s_t)$ are obtained to compute the surrogate loss $L(\theta)$, as previously defined. This surrogate loss plays a crucial role in the GAPPO algorithm by ensuring that policy updates remain stable while allowing for effective exploration. P_i is evaluated in the environment to obtain fitness score F_i using

$$F_i = \alpha \cdot \frac{R_i}{R_{max}} + \beta \cdot \frac{H_i}{H_{max}} - \gamma \cdot \frac{E_i}{E_{max}}, \tag{12}$$

where R_i is the cumulative reward obtained by the individual P_i, H_i is the policy entropy encouraging exploration, and E_i is the computational resource cost. The factors $\alpha, \beta,$ and γ are weights balancing these components.

To ensure the stability of the training process, PPO employs a clipped objective function for policy updates, which is expressed as

$$\begin{aligned} L_1 &= \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\hat{A}_t, \\ L_2 &= \text{clip}\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t, \\ L(\theta) &= \min(L_1, L_2), \end{aligned} \tag{13}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ denotes the ratio of the new and old policies, and ϵ controls the permissible range of this ratio. The clip function clip$(\cdot)$ is expressed as

$$\text{clip}(x, 1-\epsilon, 1+\epsilon) = \begin{cases} 1-\epsilon & \text{if } x < 1-\epsilon \\ x & \text{if } 1-\epsilon \le x \le 1+\epsilon \\ 1+\epsilon & \text{if } x > 1+\epsilon \end{cases}. \tag{14}$$

This objective function (14) restricts the magnitude of policy updates, thereby preventing significant shifts in the policy and enhancing training stability. Furthermore, incorporating a GA can optimize policy exploration. In each generation, the GA evaluates the fitness of multiple strategies, selecting the best-performing individuals for crossover and mutation, which generates a new set of strategies. This algorithm effectively expands the policy space and enhances the diversity and global optimality of the search.

Additionally, the value loss L_V is calculated as

$$L_V = \frac{1}{2}(R_t - V(s_t))^2. \tag{15}$$

The total loss L is computed by combining the surrogate loss and the value loss, and the network parameters given as θ are updated through gradient descent.

As the generations progress, our proposed GAPPO algorithm evaluates each individual in the population to obtain fitness scores F_i. Based on those scores, a tournament selection strategy is employed to choose individuals for reproduction. A new population P' is generated through crossover and mutation, and the original population P is updated accordingly. This evolutionary process continues until the maximum number of generations is reached.

The enhanced PPO agent with the genetic algorithm enhances task allocation in multi-agent systems by effectively balancing exploration and exploitation while ensuring diversity within the agent population. This algorithm demonstrates robustness and convergence, even in scenarios with varying numbers of agents. The operational flow of the algorithm is encapsulated in the procedure outlined in Algorithm 1.

Algorithm 1 Enhanced PPO agent with genetic algorithm

Initialize: Population of Actor–Critic networks $P = \{P_1, P_2, \ldots, P_N\}$
Set hyperparameters: learning rate lr, discount factor γ, clip ratio ϵ, and KL divergence target KL_{target}
Initialize replay buffer with capacity $C = 10,000$
Set generation counter $g = 0$
while training **do**
    for each individual $P_i \in P$ **do**
        Initialize trajectory T and observe initial state s_0
        for each time step $t = 0$ to T_{max} **do**
            Select action a_t using policy P_i based on s_t
            Execute a_t, observe reward r_t and next state s_{t+1}
            Append transition (s_t, a_t, r_t, s_{t+1}) to trajectory T
            if terminal state reached **then**
                Break
            end if
        end for
        Store trajectory T in replay buffer
    end for
    if len(replay buffer) $\geq$ BATCH_SIZE **then**
        Sample a batch of transitions from replay buffer
        for each transition in the batch **do**
            Compute returns R_t using γ
            Compute advantages $\hat{A}_t$ using Generalized Advantage Estimation (GAE)
            Get action probabilities $\pi(a_t|s_t)$ and state values $V(s_t)$
            Calculate surrogate loss $L(\theta)$ from (13)
            Calculate value loss from (15)
            Compute total loss $L = L(\theta) + L_V$
            Update network parameters θ using gradient descent
        end for
    end if
end while
while $g <$ max_generations **do**
    for each individual $P_i \in P$ **do**
        Evaluate P_i in the environment to obtain fitness score F_i
    end for
    Select individuals based on fitness using a selection strategy (e.g., tournament selection)
    Generate new population $P' = \{P'_1, P'_2, \ldots, P'_N\}$ via crossover and mutation
    Update population $P \leftarrow P'$ and increment generation counter g
end while
Periodically: Update target networks to stabilize training

This optimization framework leverages genetic operations—selection, mutation, and recombination—enabling agents to evolve strategies that better adapt to dynamic task environments. By integrating the genetic algorithm with the Actor–Critic network, agents are equipped to refine their strategies through both gradient-based and evolutionary search, enhancing adaptability and ensuring efficient coverage and resource allocation in complex and variable environments.

This section provides a comprehensive overview of the enhanced PPO agent with the genetic algorithm, detailing its initialization, iterative interactions between agents and the environment, and the strategy update phase. The distinctive features of this algorithm in the context of multi-agent task allocation are emphasized, highlighting its potential for improving operational efficiency.

4.2. Robustness Under Failure Conditions

GAPPO is designed to operate reliably in dynamic environments and addresses potential failure conditions inherent in multi-agent systems. In the event of communication breakdowns, GAPPO's decentralized architecture enables each agent to rely on local observations and previously learned policies. This design ensures that task execution can continue even when global communication is unavailable.

These design choices are based on well-established principles of decentralized control, which support system resilience under adverse conditions. Although our current experimental evaluation focuses on standard operating scenarios, the theoretical foundation of GAPPO provides a promising basis for robustness in real-world applications.

5. Experimental Results

In this section, we evaluate the established model alongside the proposed GAPPO algorithm through numerical simulations under varying experimental configurations. Specifically, four experiments were conducted to assess the performance of GAPPO in task allocation, considering scenarios with different numbers of agents and tasks. The results demonstrate that GAPPO consistently outperformed other representative algorithms, achieving efficient task allocations and maintaining robust performance across diverse conditions.

To provide additional insights into the algorithm's effectiveness, a bar chart was employed to compare energy consumption across the scenarios. The findings highlight GAPPO's capability to optimize task allocation while ensuring effective resource utilization. Overall, the evaluation underscores the practicality of GAPPO in addressing complex task allocation problems in multi-agent systems, further validating its potential to meet diverse system requirements.

Task Allocation

This subsection focuses on evaluating the effectiveness of our proposed algorithm in environment. The GAPPO algorithm is compared with the following representative algorithms.

1. Advantage Actor–Critic (A2C) [41]: The A2C algorithm optimizes decision-making by integrating policy and value functions. The advantage function $A(s_t, a_t)$ is expressed as
$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t), \tag{16}$$
where $Q(s_t, a_t)$ denotes the expected return from action a_t in state s_t, and $V(s_t)$ provides the baseline value of s_t. This structure enhances stability in learning by reducing gradient variance.
2. Proximal Policy Optimization (PPO) [42]: PPO is an on-policy algorithm designed to balance exploration and exploitation by constraining policy updates, which limits excessive updates and enhances stability and sample efficiency in learning.

3. Deep Q-Network (DQN) [43]: DQN is a value-based reinforcement learning algorithm that approximates the optimal action–value function $Q(s, a)$ using deep neural networks. Leveraging experience replay and a target network, DQN stabilizes learning and updates the Q-values as

$$\begin{aligned} \delta &= r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t), \\ Q(s_t, a_t) &\leftarrow Q(s_t, a_t) + \alpha\, \delta, \end{aligned} \tag{17}$$

   where α is the learning rate, r_t is the reward, and γ is the discount factor.
4. Deep Deterministic Policy Gradient (DDPG) [44]: DDPG is an off-policy algorithm that addresses continuous action spaces by combining an Actor–Critic framework with deterministic policy gradients. The actor network directly optimizes the policy, while the critic network evaluates the action–value function, with the policy gradient update given by

$$\nabla_\theta J \approx \mathbb{E}_{s_t \sim \rho^\pi} \left[\nabla_a Q(s_t, a|\theta^Q) \nabla_\theta \mu(s_t|\theta^\mu) \right], \tag{18}$$

   where $Q(s, a|\theta^Q)$ is the action–value function, $\mu(s_t|\theta^\mu)$ is the policy, and θ^μ and θ^Q are the parameters of the actor and critic networks, respectively.

Four scenarios are examined: (i) 100 agents are allocated to 20 tasks; (ii) 300 agents are allocated to 30 tasks; (iii) 500 agents are allocated to 30 tasks; (iv) 500 agents are allocated to 50 tasks.

Monte Carlo simulations are conducted for each scenario, with results generated from over 100 runs per scenario.

In each simulation trial, agents are randomly allocated, with an initial random allocation conducted for each agent at the starting moment, and the maximum number of iterations is set to 200 for the first scenario, while it is set to 300 for the subsequent three scenarios. The time unit Δt is defined as 1 hour (h) across all scenarios.

To evaluate reinforcement learning algorithms, we compare GAPPO, A2C, PPO, DQN, and DDPG across four increasingly complex scenarios, where agent capacity and task load are balanced, and the optimal completion time remains 1 unit time. The results demonstrate GAPPO's superior performance, rapid convergence, and stability in large-scale task allocation.

In the first scenario (Figure 2), GAPPO quickly minimizes completion time and stabilizes at a low value, whereas other algorithms show slower learning and fluctuations. As complexity increases in the second scenario (Figure 3), GAPPO achieves optimal performance with fewer iterations, while competing algorithms exhibit erratic learning curves.

In the third scenario (Figure 4), GAPPO maintains robust performance with rapid convergence and minimal fluctuations, while other algorithms struggle with instability. In the fourth scenario (Figure 5), GAPPO continues to excel, reaching optimal performance quickly despite increased task volume, while alternative algorithms progress slowly and experience frequent regressions.

Across all scenarios, GAPPO consistently reduces maximum completion time, demonstrating strong adaptability and efficiency in handling large-scale multi-agent task allocation problems. Its rapid convergence and stability establish it as a highly effective solution for dynamic reinforcement learning tasks in complex environments.

In conclusion, GAPPO outperforms other algorithms in scalability and efficiency, making it well suited for real-world applications that require fast, reliable task allocation in complex, dynamic settings.

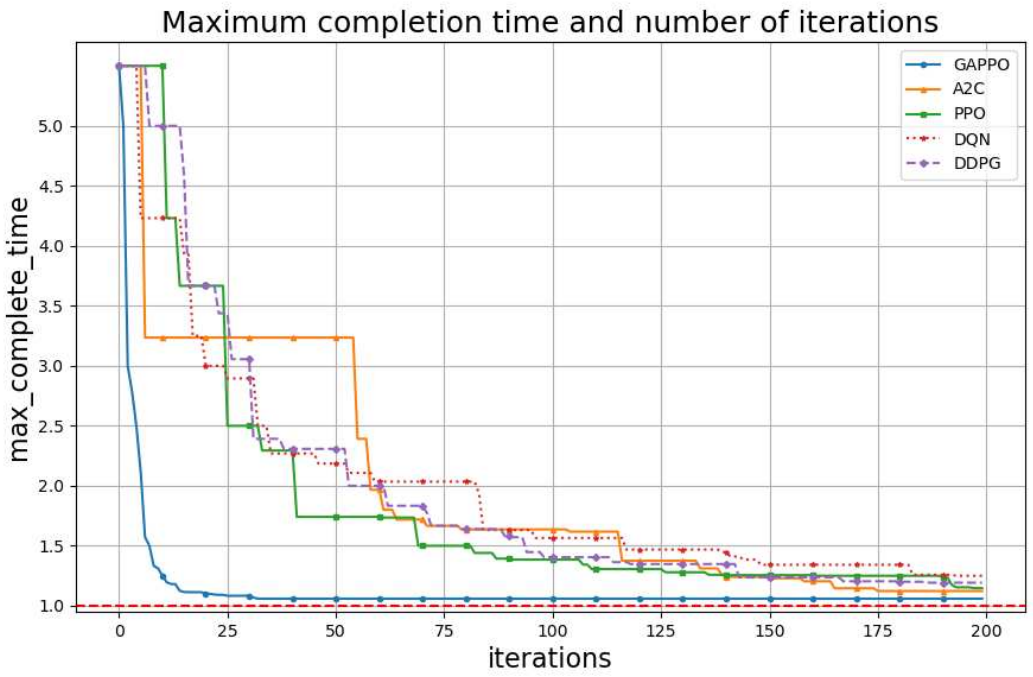


Figure 2. Convergence performance of five algorithms in the environment with 100 agents and 20 tasks. The pink dotted line indicates the value of the optimal solution, which is 1 h.

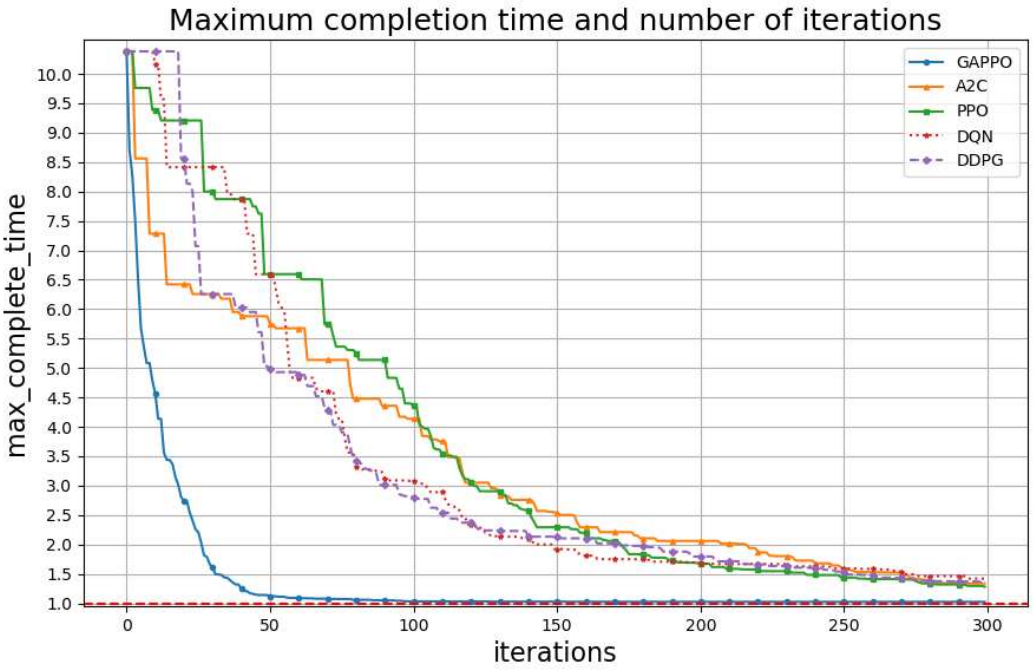


Figure 3. Convergence performance of five algorithms in the environment with 300 agents and 30 tasks. The pink dotted line indicate the value of the optimal solution, which is 1 h.

GAPPO demonstrates a significant advantage in energy efficiency for multi-agent task allocation. As shown in Figure 6, it consistently outperforms other algorithms in battery retention. In a setup with 100 agents and 20 tasks, GAPPO maintained 91.9% battery by the end of the process, while A2C, PPO, DQN, and DDPG retained only 52.2–55.5%. This notable gap, observed under identical conditions, underscores GAPPO's superior energy management without compromising task allocation performance.

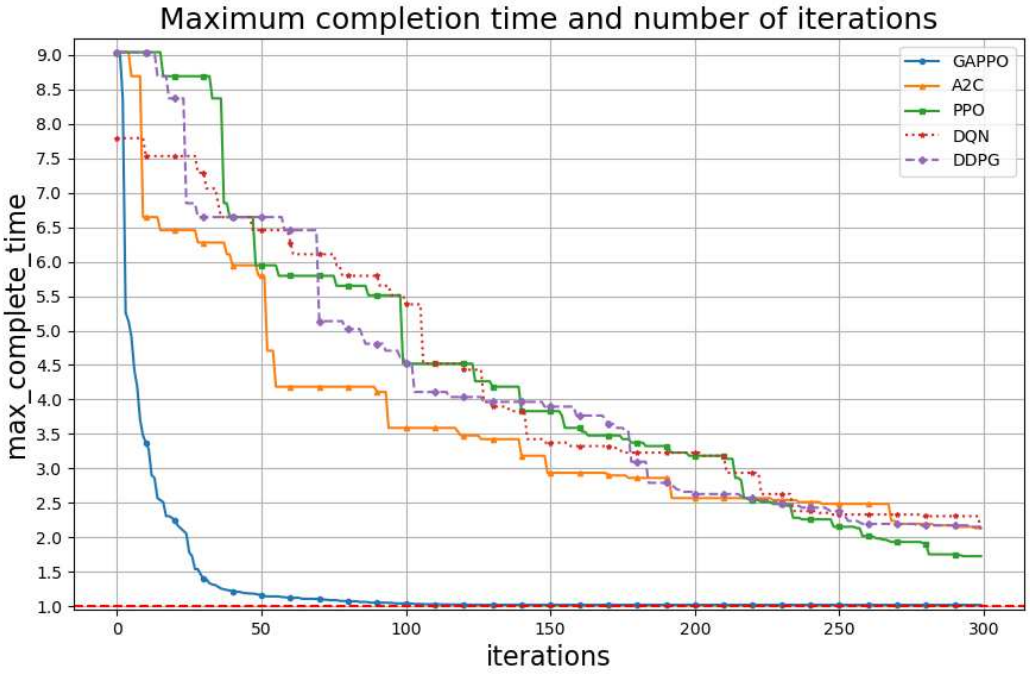


Figure 4. Convergence performance of five algorithms in the environment with 500 agents and 30 tasks. The pink dotted line indicates the value of the optimal solution, which is 1 h.

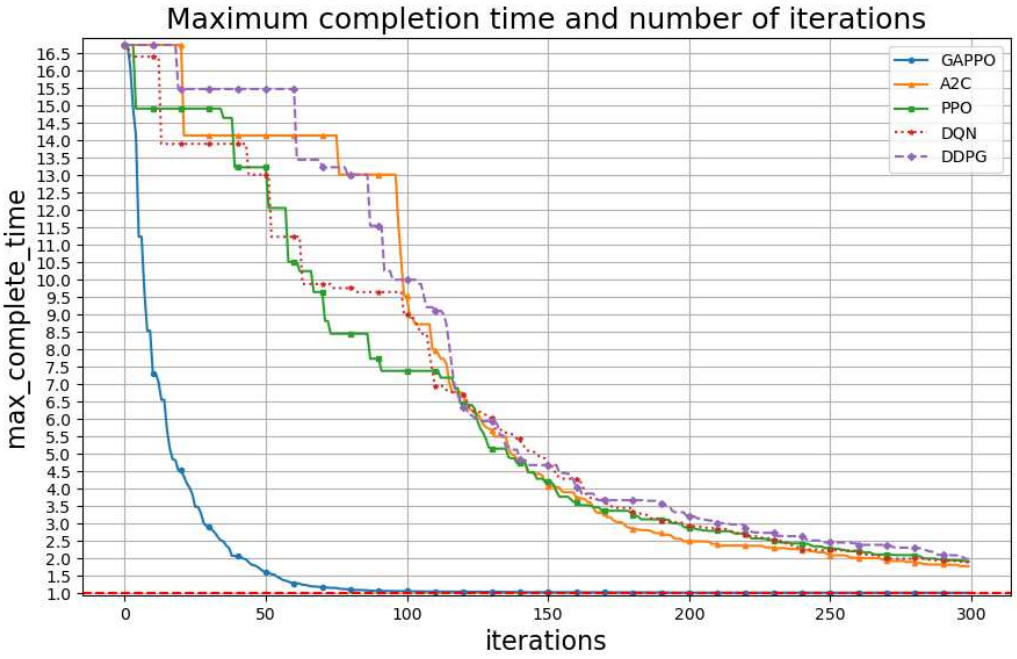


Figure 5. Convergence performance of five algorithms in the environment with 500 agents and 50 tasks. The pink dotted line indicates the value of the optimal solution, which is 1 h.

Each agent's battery level is initialized at 20, and the reduction in battery power is proportional to the distance traveled. Therefore, the battery consumption is represented as

$$E_i^{k+1} = E_i^k - k \cdot d_{ij}, \tag{19}$$

where k is a proportionality constant representing the battery consumption rate per unit distance.

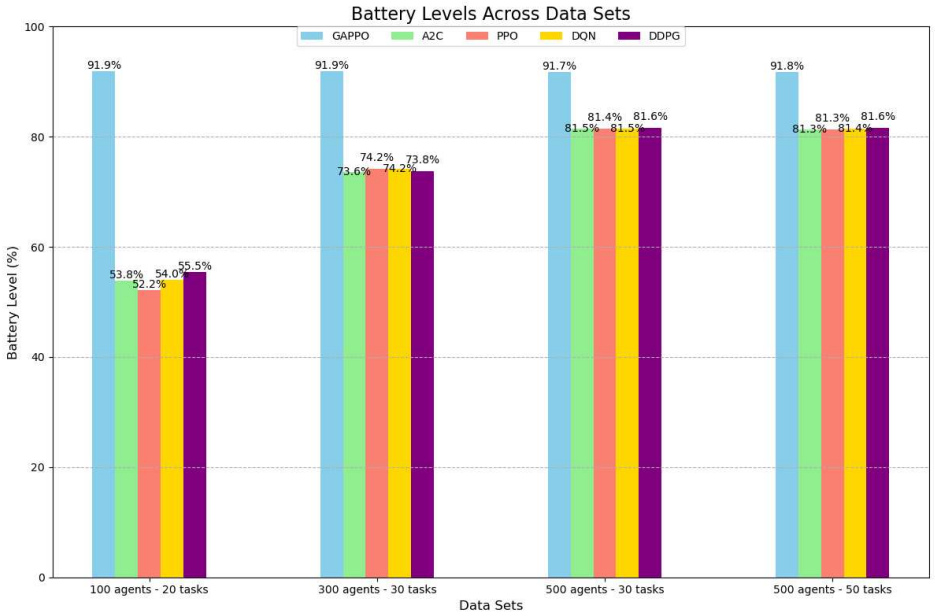


Figure 6. Comparison of five algorithms on power remaining percentage after task completion.

In larger-scale setups with 300 and 500 agents, the initial battery capacity was increased from 20 to 30 units to match the increased agent count. Despite this, GAPPO maintained a clear edge in energy management. Its battery percentage remained consistently high, between 91.7% and 91.8%, while A2C, PPO, DQN, and DDPG ranged from 74.2% to 81.6%. This sustained advantage, even with a higher initial battery capacity and increased task load, highlights GAPPO's scalability and robustness in energy optimization.

While the primary focus of GAPPO is to optimize energy efficiency in multi-agent systems, its broader implications must also be considered. One key aspect is the fairness of resource allocation, as reinforcement learning-based policies may inadvertently favor specific agents or regions if not properly designed. Ensuring that GAPPO maintains balanced allocations across all agents is crucial to prevent systemic biases in real-world applications. From a sustainability perspective, our algorithm reduces energy consumption, which is particularly beneficial for large-scale deployments, such as smart grids or autonomous fleets. However, it is important to consider the trade-offs between computational cost and energy savings. Training reinforcement learning models requires significant computational resources, and optimizing this aspect remains an open challenge for sustainable AI development. Future research could explore strategies such as adaptive model complexity or decentralized learning to further minimize computational overhead while maintaining performance.

Overall, the results demonstrate that GAPPO effectively manages energy consumption across different scales and battery configurations while maintaining performance as task complexity and agent numbers increase. Its ability to balance task allocation and energy efficiency makes it well suited for real-world applications with constrained computational and energy resources. This robustness highlights its potential for deployment in complex multi-agent environments requiring high efficiency across multiple dimensions.

In the conducted experiments, GAPPO outperformed the baseline algorithms, including PPO, DDPG, DQN, and A2C, across various metrics such as task completion time, resource efficiency, and scalability. GAPPO's superior performance is attributed to its ability to combine the strengths of both reinforcement learning and evolutionary algorithms. The genetic algorithm component enables a more diverse exploration of the policy

space, leading to more robust solutions, particularly in dynamic environments where task configurations and agent capabilities vary over time.

Compared to PPO and A2C, which rely solely on policy gradient algorithms, GAPPO benefits from the genetic algorithm's ability to select and evolve the most promising policies across generations. This evolution not only improves the exploration of the solution space but also ensures better convergence, particularly in complex, multi-agent environments where standard reinforcement learning algorithms often struggle to balance exploration and exploitation.

DDPG and DQN, being value-based algorithms, face limitations in environments with continuous action spaces or complex task dependencies. Those algorithms are more prone to getting stuck in local optima due to their reliance on fixed Q-values or deterministic policies. In contrast, GAPPO overcomes those limitations by combining both policy gradient and evolutionary search. This flexibility allows it to dynamically adjust its policies to adapt to changes in the environment and task complexity, leading to improved performance in a wider range of scenarios.

In summary, GAPPO's integration of a genetic algorithm with reinforcement learning enables it to effectively balance exploration and exploitation, outperforming traditional algorithms like PPO, DDPG, DQN, and A2C in terms of scalability, adaptability, and robustness. This makes GAPPO particularly well suited for dynamic multi-agent systems, where environment complexity and agent interactions are key factors for success.

6. Conclusions and Future Work

In conclusion, the GAPPO algorithm stands out as a robust solution to the challenges inherent in multi-agent task allocation within complex environments. Traditional algorithms often encounter significant limitations in scalability and adaptability, struggling to manage the intricacies associated with numerous agents and tasks efficiently. In contrast, the GAPPO algorithm effectively addresses these shortcomings by prioritizing energy efficiency and optimal resource distribution, leading to superior performance outcomes. Our comprehensive experimental results consistently demonstrate that GAPPO algorithm significantly reduces task completion time while maintaining high energy levels across agent fleets. This effectiveness highlights the algorithm's capability to enhance operational efficiency in diverse scenarios, solidifying its role in advancing multi-agent systems.

Future research will concentrate on several pivotal points: (1) optimizing energy management strategies within GAPPO, particularly in scenarios with varying task loads and agent configurations; (2) validating GAPPO's performance in real-world applications, such as UAV fleets or autonomous robotic systems, to ensure its effectiveness and reliability outside of controlled environments; (3) although GAPPO effectively handles communication breakdowns and unexpected agent behaviors through the strategies outlined above, there is room for improvement. Future work could explore more sophisticated fault-tolerant mechanisms, such as decentralized learning or collaborative filtering techniques, to enhance agent cooperation in extreme failure conditions. Additionally, adaptive strategies could be further refined to allow for proactive anomaly detection, enabling the system to anticipate potential failures before they disrupt performance.

Author Contributions: Software, F.Y.; Validation, T.M.; Formal analysis, J.H.; Investigation, Z.F.; Resources, T.M. and J.H.; Data curation, Z.F. and Z.N.; Writing—original draft, Z.F. and T.M.; Writing—review & editing, J.H., Z.N. and F.Y.; Project administration, J.H.; Funding acquisition, T.M. All authors have read and agreed to the published version of the manuscript.

Funding: This work is partially supported by the high-level talent fund No. 22-TDRCJH-02-013.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Chen, J.; Guo, Y.; Qiu, Z.; Xin, B.; Jia, Q.-S.; Gui, W. Multiagent dynamic task assignment based on forest fire point model. *IEEE Trans. Autom. Sci. Eng.* **2022**, *19*, 833–849. [CrossRef]
2. Liu, S.; Feng, B.; Bi, Y.; Yu, D. An Integrated Approach to Precedence-Constrained Multi-Agent Task Assignment and Path Finding for Mobile Robots in Smart Manufacturing. *Appl. Sci.* **2024**, *14*, 3094. [CrossRef]
3. Huang, L.; Wu, Y.; Tempini, N. A Knowledge Flow Empowered Cognitive Framework for Decision Making with Task-Agnostic Data Regulation. *IEEE Trans. Artif. Intell.* **2024**, *5*, 2304–2318. [CrossRef]
4. Zhuang, H.; Lei, C.; Chen, Y.; Tan, X. Cooperative Decision-Making for Mixed Traffic at an Unsignalized Intersection Based on Multi-Agent Reinforcement Learning. *Appl. Sci.* **2023**, *13*, 5018. [CrossRef]
5. Wu, J.; Li, D.; Yu, Y.; Gao, L.; Wu, J.; Han, G. An attention mechanism and adaptive accuracy triple-dependent MADDPG formation control method for hybrid UAVs. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 11648–11663. [CrossRef]
6. Yu, Y.; Zhai, Z.; Li, W.; Ma, J. Target-Oriented Multi-Agent Coordination with Hierarchical Reinforcement Learning. *Appl. Sci.* **2024**, *14*, 7084. [CrossRef]
7. Wu, J.; Zhang, J.; Sun, Y.; Li, X.; Gao, L.; Han, G. Multi-UAV collaborative dynamic task allocation method based on ISOM and attention mechanism. *IEEE Trans. Veh. Technol.* **2024**, *73*, 6225–6235. [CrossRef]
8. Li, W.; Wang, X.; Jin, B.; Luo, D.; Zha, H. Structured Cooperative Reinforcement Learning with Time-Varying Composite Action Space. *IEEE Trans. Pattern Anal. Mach. Intell.* **2022**, *44*, 8618–8634. [CrossRef]
9. Furch, A.; Lippi, M.; Carpio, R.F.; Gasparri, A. Route optimization in precision agriculture settings: A multi-Steiner TSP formulation. *IEEE Trans. Autom. Sci. Eng.* **2023**, *20*, 2551–2568. [CrossRef]
10. Fatemidokht, H.; Rafsanjani, M.K.; Gupta, B.B.; Hsu, C.-H. Efficient and secure routing protocol based on artificial intelligence algorithms with UAV-assisted for vehicular ad hoc networks in intelligent transportation systems. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 4757–4769. [CrossRef]
11. Gong, T.; Zhu, L.; Yu, F.R.; Tang, T. Edge Intelligence in Intelligent Transportation Systems: A Survey. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 8919–8944. [CrossRef]
12. Ribeiro, R.G.; Cota, L.P.; Euzebio, T.A.M.; Ramírez, J.A.; Guimarães, F.G. Unmanned aerial vehicle routing problem with mobile charging stations for assisting search and rescue missions in post-disaster scenarios. *IEEE Trans. Syst. Man Cybern. Syst.* **2021**, *52*, 6682–6696. [CrossRef]
13. Wang, H.; Wang, C.; Zhou, K.; Liu, D.; Zhang, X.; Cheng, H. TEBChain: A trusted and efficient blockchain-based data sharing scheme in UAV-assisted IoV for disaster rescue. *IEEE Trans. Netw. Serv. Manag.* **2024**, *21*, 4119–4130. [CrossRef]
14. Sampedro, C.; Rodriguez-Ramos, A.; Bavle, H.; Carrio, A.; De la Puente, P.; Campoy, P. A fully-autonomous aerial robot for search and rescue applications in indoor environments using learning-based techniques. *J. Intell. Robot. Syst.* **2019**, *95*, 601–627. [CrossRef]
15. Meng, W.; He, Z.; Su, R.; Yadav, P.K.; Teo, R.; Xie, L. Decentralized multi-UAV flight autonomy for moving convoys search and track. *IEEE Trans. Control Syst. Technol.* **2017**, *25*, 1480–1487. [CrossRef]
16. Liu, Z.; Qiu, C.; Zhang, Z. Sequence-to-Sequence Multi-Agent Reinforcement Learning for Multi-UAV Task Planning in 3D Dynamic Environment. *Appl. Sci.* **2022**, *12*, 12181. [CrossRef]
17. Wu, G.; Liu, Z.; Fan, M.; Wu, K. Joint task offloading and resource allocation in multi-UAV multi-server systems: An attention-based deep reinforcement learning approach. *IEEE Trans. Veh. Technol.* **2024**, *73*, 11964–11978. [CrossRef]
18. Guo, H.; Wang, Y.; Liu, J.; Liu, C. Multi-UAV cooperative task offloading and resource allocation in 5G advanced and beyond. *IEEE Trans. Wireless Commun.* **2024**, *23*, 347–359. [CrossRef]
19. Liu, D.; Dou, L.; Zhang, R.; Zhang, X.; Zong, Q. Multi-agent reinforcement learning-based coordinated dynamic task allocation for heterogeneous UAVs. *IEEE Trans. Veh. Technol.* **2023**, *72*, 4372–4383. [CrossRef]
20. Zhao, N.; Ye, Z.; Pei, Y.; Liang, Y.-C.; Niyato, D. Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing. *IEEE Trans. Wireless Commun.* **2022**, *21*, 6949–6960. [CrossRef]
21. Chen, R.; Li, W.; Yang, H. A deep reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for the job-shop scheduling problem. *IEEE Trans. Ind. Inform.* **2023**, *19*, 1322–1331. [CrossRef]
22. Wang, L.; Zhang, H.; Guo, S.; Yuan, D. Deployment and association of multiple UAVs in UAV-assisted cellular networks with the knowledge of statistical user position. *IEEE Trans. Wireless Commun.* **2022**, *21*, 6553–6567. [CrossRef]

23. Dai, Z.; Zhang, Y.; Zhang, W.; Luo, X.; He, Z. A multi-agent collaborative environment learning method for UAV deployment and resource allocation. *IEEE Trans. Signal Inf. Process. Netw.* **2022**, *8*, 120–130. [CrossRef]
24. Shabanighazikelayeh, M.; Koyuncu, E. Optimal placement of UAVs for minimum outage probability. *IEEE Trans. Veh. Technol.* **2022**, *71*, 9558–9570. [CrossRef]
25. Consul, P.; Budhiraja, I.; Garg, D.; Kumar, N.; Singh, R.; Almogren, A.S. A hybrid task offloading and resource allocation approach for digital twin-empowered UAV-assisted MEC network using federated reinforcement learning for future wireless network. *IEEE Trans. Consum. Electron.* **2024**, *70*, 3120–3130. [CrossRef]
26. Wang, N.; Liang, X.; Li, Z.; Hou, Y.; Yang, A. PSE-D model-based cooperative path planning for UAV and USV systems in antisubmarine search missions. *IEEE Trans. Aerosp. Electron. Syst.* **2024**, *60*, 6224–6240. [CrossRef]
27. Al-Hussaini, S.; Gregory, J.M.; Gupta, S.K. Generating Task Reallocation Suggestions to Handle Contingencies in Human-Supervised Multi-Robot Missions. *IEEE Trans. Autom. Sci. Eng.* **2024**, *21*, 367–381. [CrossRef]
28. Raja, G.; Anbalagan, S.; Ganapathisubramaniyan, A.; Selvakumar, M.S.; Bashir, A.K.; Mumtaz, S. Efficient and secured swarm pattern multi-UAV communication. *IEEE Trans. Veh. Technol.* **2021**, *70*, 7050–7058. [CrossRef]
29. Liu, D.; Wang, J.; Xu, K. Task-driven relay assignment in distributed UAV communication networks. *IEEE Trans. Veh. Technol.* **2019**, *68*, 11003–11017. [CrossRef]
30. Ren, Y.; Wang, Q.; Duan, Z. Optimal Distributed Leader-Following Consensus of Linear Multi-Agent Systems: A Dynamic Average Consensus-Based Approach. *IEEE Trans. Circuits Syst. II Express Briefs* **2022**, *69*, 1208–1212. [CrossRef]
31. Luo, Q.; Liu, S.; Wang, L.; Tian, E. Privacy-Preserved Distributed Optimization for Multi-Agent Systems With Antagonistic Interactions. *IEEE Trans. Circuits Syst. I Regul. Pap.* **2023**, *70*, 1350–1360. [CrossRef]
32. Zhang, M.; Pan, C. Hierarchical Optimization Scheduling Algorithm for Logistics Transport Vehicles Based on Multi-Agent Reinforcement Learning. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 3108–3117. [CrossRef]
33. Zhou, J.; Lv, Y.; Wen, C.; Wen, G. Solving Specified-Time Distributed Optimization Problem via Sampled-Data-Based Algorithm. *IEEE Trans. Netw. Sci. Eng.* **2022**, *9*, 2747–2758. [CrossRef]
34. Mao, X.; Wu, G.; Fan, M.; Cao, Z.; Pedrycz, W. DL-DRL: A double-level deep reinforcement learning approach for large-scale task scheduling of multi-UAV. *IEEE Trans. Autom. Sci. Eng.* **2024**, *22*, 1028–1044. [CrossRef]
35. Wang, Y.; He, Y.; Yu, F.R.; Lin, Q.; Leung, V.C.M. Efficient resource allocation in multi-UAV assisted vehicular networks with security constraint and attention mechanism. *IEEE Trans. Wireless Commun.* **2023**, *22*, 4802–4813. [CrossRef]
36. Ning, N.; Ji, H.; Wang, X.; Ngai, E.C.H.; Guo, L.; Liu, J. Joint optimization of data acquisition and trajectory planning for UAV-assisted wireless powered Internet of Things. *IEEE Trans. Mob. Comput.* **2024**, *24*, 1016–1030. [CrossRef]
37. Xu, X.; Feng, G.; Qin, S.; Liu, Y.; Sun, Y. Joint UAV deployment and resource allocation: A personalized federated deep reinforcement learning approach. *IEEE Trans. Veh. Technol.* **2024**, *73*, 4005–4018. [CrossRef]
38. Ren, W.; Beard, R.W. Consensus seeking in multi-agent systems under dynamically changing interaction topologies. *IEEE Trans. Autom. Control* **2005**, *50*, 655–661. [CrossRef]
39. Low, C.B. A dynamic virtual structure formation control for fixed-wing UAVs. In Proceedings of the 2011 9th IEEE International Conference on Control and Automation (ICCA), Santiago, Chile, 19–21 December 2011; pp. 627–632.
40. Balch, T.; Arkin, R.C. Behavior-based formation control for multi-robot teams. *IEEE Trans. Robot. Autom.* **1998**, *14*, 926–939. [CrossRef]
41. Konda, V.; Tsitsiklis, J. Actor-critic algorithms. *Adv. Neural Inf. Process. Syst.* **1999**, *12* , 1008–1014.
42. Zhang, H.; Jiang, M.; Liu, X.; Wen, X.; Wang, N.; Long, K. PPO-based PDACB traffic control scheme for massive IoV communications. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 1116–1125. [CrossRef]
43. Mnih, V.; Kavukcuoglu, K.; Silver, D. Human-level control through deep reinforcement learning. *Nature* **2015**, *518*, 529–533. [CrossRef] [PubMed]
44. Zheng, K.; Jia, X.; Chi, K.; Liu, X. DDPG-based joint time and energy management in ambient backscatter-assisted hybrid underlay CRNs. *IEEE Trans. Commun.* **2023**, *71*, 441–456. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.